

FinInsights

A KPI intelligence-to-action engine for a bank — and the bank that feeds it.

Every bank already has dashboards. What it does not have is an *answer*. A chart shows that KYC completions fell 18% last week; someone then spends a day establishing whether the drop is real or a data glitch, which segment it came from, whether it will continue, whether a campaign caused it, and what to do. The last five minutes of that day produce the summary the business actually reads. FinInsights is an attempt at the other seven hours and fifty-five minutes.

It watches a chain of five banking KPIs. For each one it checks whether the underlying data is even fit to answer from, decides whether a movement is real rather than noise, finds the segment it concentrated in, projects where it is heading, ranks whether it matters, and recommends an action with a named human owner — then writes the finding in plain English where **every figure traces back to a stored database row**. When the data cannot support a conclusion it is built to say so: it quarantines a KPI whose source is corrupted, and abstains on an ambiguous case while naming the one cheapest check that would settle it.

This repository is a **monorepo**. It contains the FinInsights platform *and* **NexaBank**, a working demo retail bank — accounts, transfers, loans, KYC, cards, four paid "Pro" modules — that produces the telemetry and the core-banking records the platform analyses. NexaBank is a data source, not the product, but it is a real Express + Prisma + Next.js application, so the analytics have something honest to chew on. NexaBank also carries the **Simulate console**, which plants anomaly scenarios *in the source data* so the engine has to discover them the way it would a real incident.

The governing document is [CLAUDE.md](#). Read it before writing code. This README covers what the system is and how to run it.

Table of contents

- [The rules that define the product](#)
- [The five KPIs](#)
- [Architecture](#)
- [The data model — Bronze, Silver, Gold](#)
- [The intelligence layer](#)
- [The LLM narrator \(Ollama / Groq / off\)](#)
- [Requirements](#)
- [Installation](#)
- [Loading data — the demo process](#)
- [The Simulate console and the demo scenarios](#)
- [Configuration reference](#)
- [Project structure](#)
- [Working on the code](#)
- [API surface](#)
- [Known limitations](#)

- [Troubleshooting](#)
- [FAQ](#)
- [Documentation map](#)
- [Maintainers and licence](#)

The rules that define the product

The golden rule

The language model is never the source of a number.

All analysis is done by deterministic tools — SQL, statistics, simple rules, light ML. A small model does exactly two things: it reads the question and plans the investigation, and it turns the finished, verified results into a sentence. Every number shown to a user traces back to a value a tool computed and stored. If a number cannot be traced, it is not shown.

Two consequences, both load-bearing:

- **Turning the model off changes no number.** Findings are still produced; they are expressed in a fixed format instead of prose. The business is never dependent on a provider staying available.
- **The model may choose the path. It may not choose the numbers.**

The other invariants

Rule	Why
Compute every number with a tool and store it; the narrator only phrases stored numbers.	An audit trail that a disputed figure can be re-checked against months later.
Read the warehouse only through the Metric API. Tools never write raw SQL against raw tables.	One place where correctness (dedup, canonicalisation, sessionisation) is applied.
Tag every number with the engine that produced it — <i>sql, stats, rules, ml, llm</i> .	The LLM-vs-non-LLM split on each insight is <i>read back</i> from records, never asserted by the model.
Choose the decomposition by the KPI's shape (rate, count, money), not a fixed formula.	This is a bank, not a shop. There is no price-volume-mix method anywhere.
Abstain when data is thin or evidence conflicts. Say what is missing and the one check.	A system that occasionally invents a plausible cause is worse than no system.
Plant anomalies in the source data first ; the engine must discover them.	The demo is honest only if the engine is never told where to look.

Refusal is a feature

- If the data is **corrupted**, the KPI is quarantined and an engineering note is issued instead of a business narrative. The movement is never dressed up as growth.
- If the evidence is **genuinely ambiguous**, the engine says so, states what is missing, and names the single cheapest check that would settle it.

- If a KPI is **too new to have a history**, it reports a wide, clearly caveated range rather than a confident forecast built on nothing.

What every insight carries

- **Evidence card** — source freshness, the method used, each driver's contribution, a confidence value, and lineage: which events, which tables, which contract.
- **Engine breakdown** — which parts were SQL, statistics, rules, ML or LLM, and which numbers came from where. Read from stored records.
- **Telemetry** — latency, model calls, tokens, and estimated cost for that insight.

The five KPIs

One connected chain, not twenty independent metrics. Each is a hand-written YAML contract in **contracts/** that declares what the number means, which additive parts it is built from, which dimensions may legitimately be sliced, who it is for, its detection thresholds, and the **closed list** of actions a recommendation may propose.

#	KPI	Shape	Source of truth	Stored fundamentals
1	New account signups	count	<code>silver.fact_account_openings</code>	<code>accounts_opened</code> by day of <code>opened_at</code>
2	KYC completion rate	rate	<code>silver.fact_loan_applications.kyc_step</code>	<code>kyc_started</code> , <code>kyc_completed</code>
3	Loan approval volume	count	<code>silver.fact_loan_applications (status = APPROVED, by decided_at)</code>	<code>loans_approved</code> , <code>principal_approved</code>
4	Revenue	money	modelled from measured inputs	<code>fee_revenue</code> , <code>interest_accrued</code> , <code>pro_revenue</code>
5	Transaction failure rate	rate	<code>silver.fact_transactions.status</code>	<code>txn_total</code> , <code>txn_failed</code>

They form a declared driver graph, so several simultaneous movements are reported as **one story with a driver chain** rather than five separate alerts:

```
signups → kyc_completion_rate → loan_approval_volume → revenue
transaction_failure_rate → (drags any of them down)
```

So "why did revenue drop?" is answered by walking back up that chain, not by a separate revenue model.

Every KPI value comes from the daily banking snapshot (`bronze.core_banking`), never the clickstream. That is a correctness decision: the batch facts carry dimensions the bank actually recorded (region, branch, channel, mcc, loan_type, risk_segment), while the clickstream carries dimensions the producer *invented*. The clickstream provides funnel-stage detail and journey reconstruction only — never a figure a reader sees as a KPI.

How a KPI is explained depends on its shape:

- A **rate** through a funnel — explained by which stage leaked and which segment drove it, always computed on the underlying counts, never on the rate itself.
- A **count** — explained by segment and by time.
- **Money** — explained by which fee line moved, and which upstream volume driver caused it.

A KPI with no contract still gets a conservative automatic treatment — it can be detected, localised and forecast — but never a causal claim or a recommended action. Promoting one to full depth is writing its contract file; no code changes.

Three personas, one truth

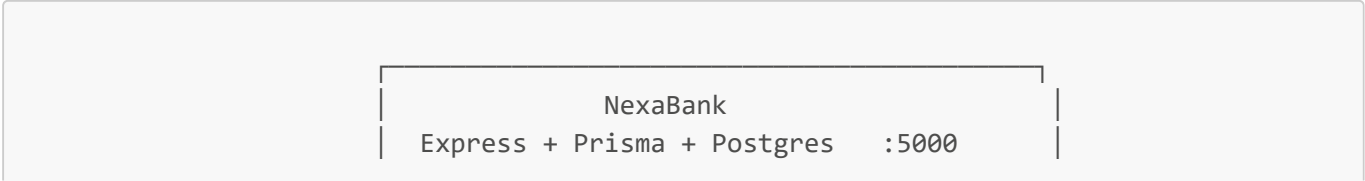
Personas are lenses on the same five KPIs. The same signal cards are built once, then filtered and reframed. Resolved **server-side** from your role; you can switch only within an allowlist, so a request can never widen access.

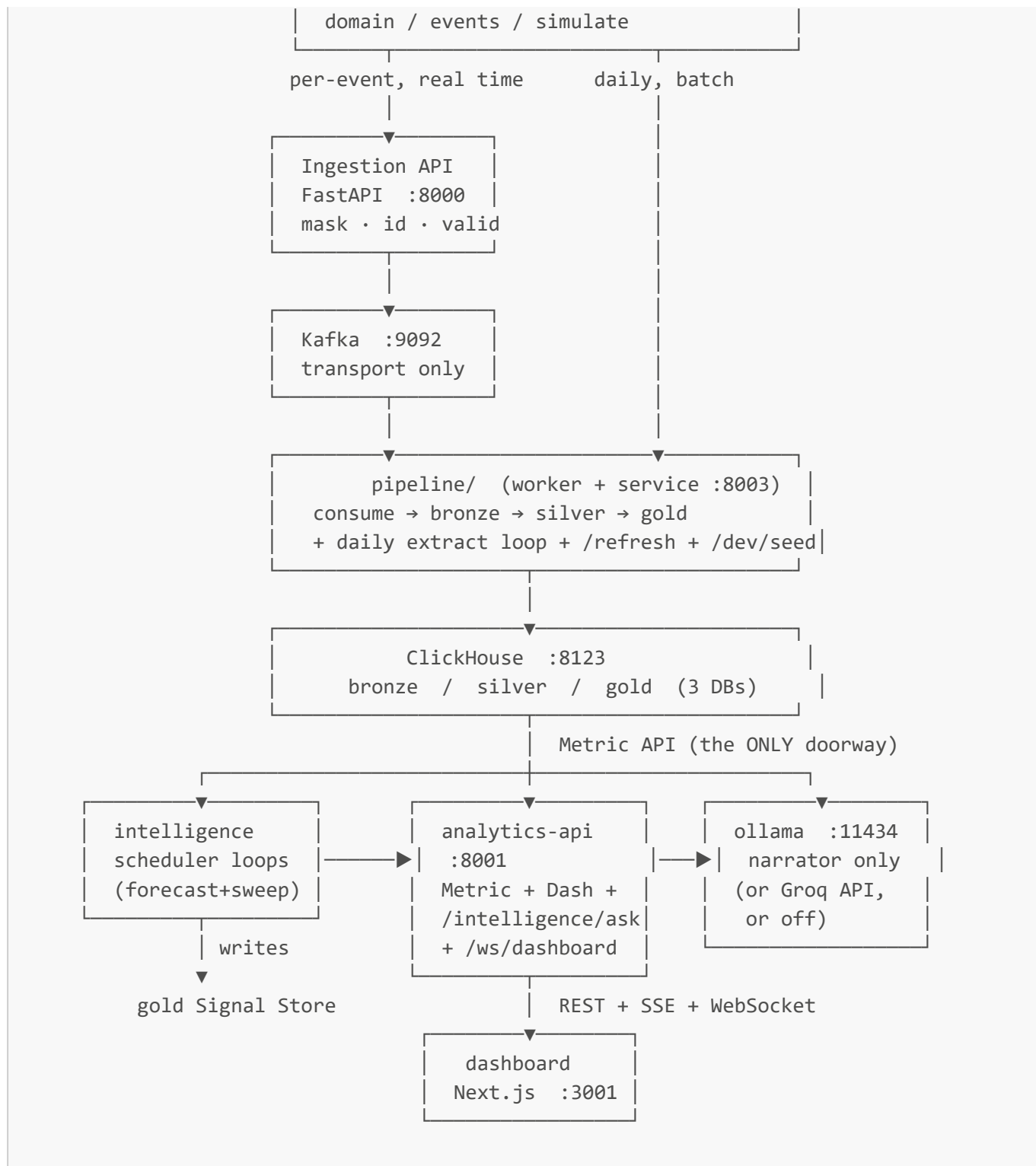
Persona	Cares about	Sees
CFO / Executive	Revenue, approval volume	The money numbers. Short, outcome-first narrative.
Ops / Onboarding Manager	Signups, KYC completion, where the funnel leaks	Operational detail — which step, which device, which vendor. No revenue.
Risk / Fraud Analyst	Transaction failures, unauthorized access	Urgent-flag-and-isolate behaviour, risk-owner actions.

`analyst` is the default persona (full method depth). Entitlement is applied **before assembly, not after**: a restricted figure is absent from a restricted reader's report rather than removed from it — and if it could be back-computed from a ratio (`revenue_per_transaction`, `revenue_at_risk`), that ratio is suppressed too. The only KPI-level restriction is: **Ops does not see revenue.**

Architecture

Five deployable systems in one repo, joined by exactly two contracts — an HTTP event envelope and a tenant-id vocabulary. No shared library, no shared database, no shared type package. Each system can be deployed and reasoned about on its own.





The containers

`docker compose up -d` starts **14 services**. `ollama` (GPU model server) starts by default; `vllm-server` is behind `--profile gpu` and `tools` behind `--profile tools`.

Service	Port	Image / build	What it does
<code>zookeeper</code>	2181	<code>cp-zookeeper:7.4.0</code>	Kafka coordination
<code>broker</code>	9092 / 29092	<code>cp-kafka:7.4.0</code>	Kafka — transport only, never the system of record

Service	Port	Image / build	What it does
init-kafka	—	cp-kafka	one-shot: creates the feature-events topic
clickhouse	8123 / 9000	clickhouse-server:24.3	the warehouse — bronze / silver / gold databases
migrate	—	root Dockerfile	one-shot: warehouse/migrate.py applies the layered DDL
ingestion-api	8000	root Dockerfile	event intake: mask PII, deterministic event id, shape-validate → Kafka
processor-worker	—	root Dockerfile	Kafka consumer → bronze.events , batch/flush, dead letters
pipeline	8003	root Dockerfile	extract loop + transform loop + POST /refresh + POST /dev/seed
intelligence	—	root Dockerfile	the scheduler: forecast batch + investigation sweep, writes the Signal Store
analytics-api	8001	root Dockerfile	Metric API + dashboard API + /intelligence/ask + /ws/dashboard
ollama	11434	omesh2004/ollama-fininsights:latest	local narrator model (Qwen 2.5 3B), GPU passthrough
vllm-server	8002	vllm/vllm-openai:latest	legacy narrator server — --profile gpu only
dashboard	3001	./dashboard	FinInsights dashboard (Next.js 15 / React 19)
nexabank-backend	5000	./nexabank/backend	the bank + the Simulate engine (Express 4 / Prisma 6 / Postgres)
nexabank-frontend	3002	./nexabank/frontend	NexaBank retail banking app (Next.js)
tools	—	root Dockerfile, --profile tools	operator scripts (seed, gates, data-quality)

Every Python service (**migrate**, **ingestion-api**, **processor-worker**, **pipeline**, **intelligence**, **analytics-api**, and **tools**) runs the **same** image, built from the root **Dockerfile**, and differs only by its compose **command**.

The two contracts between systems

1. **The event envelope** — what NexaBank POSTs to **ingestion-api:8000/events** (**docs/INTERFACES/event_envelope.md**). Ingestion validates

shape only; Silver **rejects** an unresolvable event name to the dead-letter table with a reason (it no longer coerces).

2. **The tenant vocabulary.** Round 2 is one bank. The Prisma id is **bank_a**; the analytics id is **nexabank**; the IFSC prefix is **NEXA**. The Analytics API accepts **bank_a** in any query parameter and rewrites it before RBAC runs. Several maps must agree — the dashboard feature map, the API tenant scopes, **rbac.json**, and NexaBank's **eventTracker.ts**.

Data flow, in order

1. NexaBank writes a **banking fact** (a loan application, a transaction) to Postgres **and** emits the matching behavioural event. Both, always — a fact with no clickstream is a bug.
2. **Forwarding is fire-and-forget.** Telemetry never blocks banking; forwarding outcomes are counted so the Trust Gate can tell "the KPI dropped" from "the forwarder broke".
3. **Ingestion** masks personal data, assigns a deterministic **event_id** (retries cannot double-count), shape-validates, and publishes to Kafka. If the broker is unreachable it falls back to writing ClickHouse directly, tags the row, and still returns **202** — **a silent path**; **curl localhost:8000/health** reports **ingest_path**.
4. **processor-worker** consumes into **bronze.events**.
5. **pipeline** derives **silver** then **gold** on a timer, and on demand via **POST /refresh**.
6. **The daily extract** pulls loan / account / ledger state from NexaBank's token-guarded **/api/extract/*** endpoints into **bronze.core_banking**, with its own source id and freshness watermark.
7. **The Metric API** is the only doorway into the warehouse. Named, tenant-scoped reads.
8. **The intelligence layer** calls the Metric API, writes findings to the Signal Store in **gold**, and narrates from stored rows only.

Determinism

The intelligence layer is fully deterministic — the same rows in produce byte-identical Signal Store rows out. Ids are *derived* by hashing inputs, never generated; the window is pinned once at whole UTC midnights and never recomputed inside a stage; only exact aggregates are used (**uniqExact**, **quantileExact** — never **uniq/quantile**, which are non-deterministic); every ranking has a full tiebreaker; floats are rounded at the write boundary.

The data model — Bronze, Silver, Gold

Three layers in ClickHouse, **one database per layer**. A table belongs to exactly one layer, and the layer decides who may read it. Full detail: **docs/DATA_MODEL.md**.

Layer	Database	Holds	Read by
Bronze	bronze	Raw records exactly as received. Never edited, never deleted. The audit trail.	the pipeline only
Silver	silver	Cleaned & conformed — canonical names, deduplicated by event_id , masked, sessionised, one UTC day.	the Metric API

Layer	Database	Holds	Read by
Gold	gold	Ready-to-serve KPI rollups, the segment cube, and the Signal Store.	the Metric API, the tools, the dashboard

The dashboard and the tools never read Bronze — it is a separate database and it is not in their read scope.

Bronze

Table	Grain
bronze.events	one row per clickstream event as received
bronze.core_banking	one row per extracted record, per entity, per extraction
bronze.events_dead_letter	one row per event that failed the pipeline, with the verbatim payload and reason

Plain MergeTree, partitioned by month. No ReplacingMergeTree — Bronze must not collapse anything, because a duplicate arriving twice is itself the evidence the Trust Gate reads.

Silver

Where every correctness rule is applied exactly once — canonicalise the name, deduplicate by event_id, mask, rebuild sessions, pin the day to UTC, and **reject rather than coerce** an unresolvable name. silver.fact_* are rebuilt wholesale each run (TRUNCATE + re-derive from Bronze), so a purged/replaced source row cannot persist as a stale fact.

Tables: silver.events, silver.sessions, silver.fact_transactions, silver.fact_loan_applications, silver.fact_account_openings, silver.dim_customer, silver.dim_branch, silver.dim_campaign, silver.dim_calendar, silver.dim_fee_schedule. (fact_cards, fact_campaign_interactions, dim_macro_environment are declared but not built — not in the 5-KPI chain.)

Gold

Table	Grain / purpose
gold.kpi_daily	tenant × kpi × date × fundamental — the value of every additive fundamental, per day
gold.kpi_daily_by_dim	the segment cube Localize searches
gold.funnel_daily	tenant × funnel × date × stage — so a rate is always derived from counts

Rollups store additive fundamentals, never rates. A rate is computed at read time from its two counts; storing it makes it non-additive, and a non-additive quantity cannot be localised.

The **Signal Store** (also in gold) is where every tool writes what it concluded and the only thing the narrator may read: investigations, trust_findings (including passes), anomalies, root_causes, forecasts, causal_effects, recommendations, insights,

`model_runs`, `outcomes`. Every row carries `investigation_id` — a Trust Gate failure produces a narrative with no anomaly, so hanging the audit trail off `anomaly_id` would lose the runs that matter most.

Measured versus fabricated dimensions

A dimension may be sliced by the intelligence layer **only if it was measured**. Geography, city, device type and channel are currently invented by the producer, at random, once per session. They are session-invariant, so contributions over them sum to 1.0 and an explanation looks complete — over dice rolls. Silver carries a per-row declaration of which attributes were fabricated, and the contract loader **refuses to localise** a dimension that appears in it (outside the seeded dataset). Ranked, confident, meaningless output is the worst failure mode this system has.

The intelligence layer

An agent with tools. A user asks a question in plain language, or a persona has a standing question. The agent reads the intent, decides which KPIs and tools to run and in what order, runs them, collects verified findings into the Signal Store, then narrates. Full detail: [docs/INTELLIGENCE.md](#).

The division of labour

Does	Who	Never does
Reads intent, plans the investigation, decides when to stop	the model	produces a number
Computes every figure	the tools	decides what to investigate
Checks every figure in the text traces to a stored row	the verifier	rewrites a figure
Phrases the finished result	the model	introduces a figure the verifier did not clear

The scheduled pipeline — six tools per KPI

The `intelligence` service runs two loops: a **forecast batch** (hourly) and an **investigation sweep** (every ~3 minutes, over the 7 / 30 / 90-day windows). For each tenant and each governed KPI the sweep asks six questions in order and records each answer as a Signal Store row.

#	Tool	Question	Method	Writes to
1	Trust Gate	Is this data fit to answer from at all?	rules + SQL	<code>trust_findings</code> (pass or fail)
2	Detect	Is the deviation real, material and persistent?	statistics — robust baseline, change-point, false-discovery control	<code>anomalies</code>

#	Tool	Question	Method	Writes to
3	Localize	Which segment is it concentrated in?	statistics + SQL (PSqueeze on large cubes)	<i>root_causes</i>
4	Forecast	Where is it heading?	classical baseline; optional small time-series model	<i>forecasts</i>
5	Materiality	Does this matter enough to surface?	rules + statistics (strength × business impact × persistence)	ranks the story chain
6	Decide	Which owned lever addresses it?	rules + <i>contracts/levers.yaml</i>	<i>recommendations</i>

Build order is not execution order: Forecast runs as a batch *before* Detect, because Detect scores against the band Forecast stored.

Detect and Localize score an additive fundamental, never a rate — a rate metric narrated as though its count were the rate ("rose 79.6% to 97.00" when the rate was 1.0 every day) is a class of bug the verifier cannot catch, because it checks that a number came from a row, not that the row is the quantity the sentence names.

Decide produces the seven-field recommendation the brief asks for: *driver, controllable lever, action, expected impact (a range, never a point), owner role, confidence, monitoring plan*. No bandits, no uplift modelling — a closed list of permitted actions per KPI, declared in the contract.

The query agent

On top of the scheduled pipeline sits a **persona-aware query agent** — a bounded *comprehend → plan → execute → observe → validate → re-plan* loop over a catalogue of deterministic tools. The planner chooses *capabilities*, never numbers; the tools produce the numbers; the verifier checks them. It works with **no model at all** (the rule-based planner is dependency-free); an optional LLM planner falls back to it on any failure. A call to an unknown tool, a tool this persona may not use, or an argument the schema does not name is refused before execution — a bad plan fails closed.

Ask it at *POST /intelligence/ask* (or */intelligence/ask/stream* for SSE). Every run records a trace: each step names the tool it called, what it passed, and how long it took.

The narrator and the verifier

The narrator writes the explanation **from the stored findings only**, per persona — raw query output is never in its context. The verifier then checks every number in the text against the claim set; if a number does not trace, the narrative is regenerated once, then falls back to a plain deterministic template. It never ships an unverified number. Phrasing is centralised so the unit a figure was measured in travels with it into every sentence, and a modelled figure carries its qualifier every time.

The feedback loop

One click per insight — *was the root cause correct, was this a known event, was the action taken*. No model retraining. The response writes a structured row that does exactly one of three things: adds a suppression / known-event rule Detect reads, adjusts a method-reliability weight used when ranking causes, or corrects a definition in a KPI contract.

The LLM narrator (Ollama / Groq / off)

The layer is **complete without a model**. It phrases English; it never produces a number. Every figure it emits is re-checked against the stored claim set, and an unverifiable draft falls back to the deterministic template. Turning it off changes no number.

Three ways to run it, selected per request (there is a model switcher in the dashboard's chat UI, persisted in `localStorage`):

Provider	How	Notes
Local Ollama (GPU) — default	the <code>ollama</code> container serves <code>qwen2.5:3b-instruct</code> on <code>:11434</code> , OpenAI-compatible	GPU passthrough via <code>deploy.resources.reservations.devices</code> . On a machine with no NVIDIA GPU , drop that block from <code>docker-compose.yml</code> (Ollama runs on CPU) or set <code>INTELLIGENCE_LLM=0</code> .
Groq / Grok API	set <code>GROQ_API_KEY</code> + <code>GROQ_MODEL</code> (e.g. <code>llama-3.3-70b-versatile</code>) in <code>.env</code> ; pick "⚡ Groq" in the chat UI, or <code>provider: "groq"</code> in the <code>/intelligence/ask</code> body	Cloud. Bearer auth is added automatically when <code>LLM_API_KEY</code> / <code>GROQ_API_KEY</code> is set.
Off	<code>INTELLIGENCE_LLM=0</code>	The deterministic template covers every demo scenario and is bit-exact.
Legacy vLLM	<code>docker compose --profile gpu up -d vllm-server</code> , point <code>VLLM_URL</code> at <code>http://vllm-server:8000/v1</code>	Kept for reference; the app default now targets <code>ollama</code> .

`VLLM_URL` (the variable name is historical) is any OpenAI-compatible base URL. No model name is hardcoded downstream — leave `INTEL_LLM_MODEL` empty and the narrator asks the server what it serves.

Confirm what actually ran:

```
docker compose exec clickhouse clickhouse-client --password clickhouse \  
  --query "SELECT engine_type, count() FROM gold.model_runs GROUP BY engine_type"
```

Requirements

Everything runs in Docker. You do **not** need Python, Node, Kafka or ClickHouse on your machine, and you should not use a host virtual environment — it drifts from the image silently.

What	Why
Docker Desktop / Engine with Compose v2 (<code>docker compose</code> , not <code>docker-compose</code>)	everything is a container
~10 GB RAM available to Docker	Kafka + Zookeeper + ClickHouse + three Next.js servers + Ollama is not a light stack
~20 GB free disk	the Python image, two <code>npm ci</code> runs, and the Ollama image (model baked in)
Git with normal line-ending handling	shell scripts and SQL are pinned to LF via <code>.gitattributes</code>

One external service you must provide: a **PostgreSQL 14+** database for NexaBank. There is deliberately no Postgres in `docker-compose.yml` — the bank's own records live outside the analytics stack, the way they would at a real customer. Wired for **Supabase** (free tier is fine), which gives the two connection strings Prisma wants; any plain PostgreSQL works too (point `DATABASE_URL` and `DIRECT_URL` at the same URL).

Optional:

- An **NVIDIA GPU** + NVIDIA Container Toolkit — only if you want the local Ollama narrator on GPU. Without one, drop the `deploy:` block from the `ollama` service, or run `INTELLIGENCE_LLM=0`, or use Groq.
- A **Google OAuth client** — the dashboard signs in with NextAuth. There is also a "Development Login" credentials provider (email only, no password) for local use, so Google is not strictly required; `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` can be placeholder strings if you only use dev login.

Ports that must be free: `2181 3001 3002 5000 8000 8001 8003 8123 9000 9092 11434 29092` — plus `8002` only with `--profile gpu`. None are configurable via env; every mapping in `docker-compose.yml` is a literal.

Installation

Budget ~20 minutes for the first run: ~10 for the image build, the rest for credentials and seeding. There is no one-shot bootstrap script.

1. Clone

```
git clone https://github.com/abhishekkumawat-47/Nucleus-Analytic-tool-with-banking-website.git
cd Nucleus-Analytic-tool-with-banking-website
```

2. Create the environment files

Compose declares `env_file`: for two files that are **gitignored and not in the repo**. Without them `docker compose up` fails immediately with an env-file error.

a. `dashboard/.env.local` — create it (there is no template):

```
GOOGLE_CLIENT_ID=<google oauth client id, or any placeholder if using dev login>
GOOGLE_CLIENT_SECRET=<google oauth secret, or any placeholder>
NEXTAUTH_URL="http://localhost:3001"
NEXTAUTH_SECRET=<a long random string>
NEXT_PUBLIC_API_URL="/api"
NEXT_PUBLIC_ANALYTICS_WS_URL="ws://localhost:8001"
NEXT_PUBLIC_NEXABANK_URL="http://localhost:3002"
```

b. `nexabank/backend/.env` — create it:

```
NODE_ENV=development
PORT=5000
FRONTEND_URL=http://localhost:3002

# Your external PostgreSQL. With Supabase: DATABASE_URL is the pooled string (port 6543),
# DIRECT_URL is the direct one (port 5432). With plain Postgres, use the same URL twice.
DATABASE_URL=postgresql://USER:PASSWORD@HOST:6543/postgres?
pgbouncer=true&sslmode=require
DIRECT_URL=postgresql://USER:PASSWORD@HOST:5432/postgres?sslmode=require

JWT_SEC=<a long random string>

# Tenant seed (read once by src/seedTenants.ts on every container start). Round 2 is one bank.
TENANT_A_ID=bank_a
TENANT_A_NAME=NexaBank
TENANT_A_IFSC=NEXA
TENANT_A_BRANCH=0001

# The built-in bank administrator. SET SYSTEM_PASSWORD before the first boot – otherwise a
# random one is generated and you cannot sign in to the admin console or the Simulate page.
SYSTEM_EMAIL=system@nexabank.internal
SYSTEM_NAME=NexaBank System
SYSTEM_TENANT=bank_a
SYSTEM_PASSWORD=<choose one now>

# Internal ledger accounts every simulated transaction books against.
ACC_PRO_LICENSE=NEXABANK-SYSTEM
ACC_REVENUE=NEXABANK-SYSTEM-REVENUE
```

```
ACC_EXTERNAL=EXTERNAL-BANK
ACC_MERCHANT=MERCHANT-ID
ACC_CRYPTOCRYPTO=EXCHANGE
ACC_WEALTH=WEALTH-REBALANCE-SYS
```

c. Root .env — optional. Compose auto-loads it for `${VAR}` interpolation, and every variable has a default (`${VAR:-default}`), so a basic run works without it. Copy the example and add the LLM keys if you want them:

```
cp .env.example .env
```

```
# .env additions
EXTRACT_API_TOKEN=local-dev-extract-token
INTELLIGENCE_LLM=1 # 0 turns the narrator off entirely
VLLM_URL=http://ollama:11434/v1 # OpenAI-compatible base URL (name is historical)

# Only if you want the Groq provider:
GROQ_API_KEY=<your groq key>
GROQ_MODEL=llama-3.3-70b-versatile
LLM_PROVIDER=ollama # default provider

# Only for the legacy vLLM GPU profile:
# HF_TOKEN=hf_...
```

3. (Optional) Grant yourself dashboard access in `rbac.json`

`rbac.json` is checked in and keyed by email. It ships with the maintainers' addresses — replace them with yours. Put your email under `app_admins.nexabank` to see the detailed pages; a `super_admin` deliberately sees only aggregate summaries. An email that appears nowhere resolves to a role that is refused everything.

If you set up a Google OAuth client, add your own address as a **Test user** on the consent screen and set the redirect URI to `http://localhost:3001/api/auth/callback/google`.

4. Build and start

```
docker compose up -d --build
```

First build is genuinely slow. Watch the two things that fail quietly:

```
docker compose logs -f init-kafka # must end with the 'feature-events'
topic created
docker compose logs -f nexabank-backend # 'prisma db push' + seedTenants must
```

```
both succeed
docker compose logs -f migrate           # must exit 0 – it applies the
bronze/silver/gold DDL
```

5. Verify the stack is healthy, not merely running

```
curl -s localhost:8000/health           # ingest_path: "kafka" (good) |
"clickhouse_fallback" (broker was down at startup)
curl -s localhost:8001/health           # {"status":"ok","metric_api":{"...}}
curl -s localhost:8003/health           # pipeline
curl -s localhost:5000/                  # NexaBank

# ClickHouse is up and the layered schema ran
docker compose exec clickhouse clickhouse-client --password clickhouse \
  --query "SHOW DATABASES"              # expect: bronze, silver, gold, default,
system
```

Interactive API docs: <http://localhost:8000/docs> (ingestion), <http://localhost:8003/docs> (pipeline). The analytics API's [/docs](#) is behind RBAC — a plain browser gets 403.

Loading data — the demo process

The stack boots empty. Every dashboard shows zeros until you generate traffic — the single most common "is it broken?" moment. The order matters and the code enforces it with HTTP 409 refusals.

Step 1 — reference data first (always)

Branches are reference data. A customer created against an empty branch table gets a null branch, which silently removes **region** from every KPI that localises on it, so the Simulate route refuses with 409 rather than allow it.

```
docker compose exec nexabank-backend npx tsx src/scripts/seedReferenceData.ts
```

Step 2 — load a dataset

Two paths. Both are legitimate; they differ in fidelity and speed.

Path A — the real pipeline (slow, high fidelity)

Drives [/events/simulate](#) through the real auth path, writing **both** telemetry and real core-banking rows to Postgres, which then flow through ingestion → Kafka → the worker → the extract → the transforms.

```
docker compose exec nexabank-backend npx tsx src/scripts/generateDemoData.ts
docker compose exec nexabank-backend npx tsx src/scripts/applyLifecycle.ts #
derive account/card/txn states
```

`generateDemoData.ts` seeds a ~55-day baseline plus a shorter recent cohort whose behaviour differs — the recent cohort is the movement Detect has to find, recorded nowhere.

Path B — the fast demo path (ClickHouse-direct, ~seconds)

The **Admin Simulate console** at <http://localhost:3002/admin/simulate> (sign in as `SYSTEM_EMAIL` / `SYSTEM_PASSWORD`) has a "**Reset demo data**" button. It wipes the fact history, mints a fresh ~5,000-customer baseline **straight into ClickHouse** via the pipeline's `/dev/seed`, and re-scores every KPI. ~35 seconds. Do this once before a demo.

```
Admin Simulation Console → "Reset demo data (clean baseline, ~60s)"
```

Under the hood this is `POST {nexabank}/api/events/simulate/reset` → `POST {pipeline}/dev/seed` with `purge_full: true`, `create_accounts: true`. The pipeline's incremental extract is watermark-guarded, so it does **not** re-flood the real history back on its next tick — only a slow `/events/simulate/plant` (a full re-extract) would.

Step 3 — sign in to the dashboard

Open <http://localhost:3001>. Sign in with the Google account in `rbac.json`, or use **Development Login** with any email you added to `rbac.json`. You land on the app selector, then a populated dashboard.

Starting over

```
# Wipe the simulated banking data in Postgres (keeps ADMIN logins + the clearing
accounts)
docker compose exec nexabank-backend npx tsx src/scripts/resetDemoData.ts --yes

# Stop everything, keep the data
docker compose down

# Stop everything and DESTROY the local volumes (ClickHouse, Kafka, Ollama model
cache)
docker compose down -v
```

`docker compose down -v` touches **nothing** in your external PostgreSQL — customers, accounts and transactions survive there. Use `resetDemoData.ts` for that side.

The Simulate console and the demo scenarios

<http://localhost:3002/admin/simulate> — admin only. Configure a population (user count, historical days), pick a **scenario template**, and run it.

Templates plant a movement **in the source data**. Each declares its own ground truth to [fixtures/planted_truth.json](#), so afterwards you can check whether the engine found what was planted — and, just as importantly, whether it found things that were not.

Template	What it plants
Baseline — no change	Rebuilds the bank with no movement. Record this as the <i>before</i> state.
KYC completion falls — one region	Onboarding leaks in Europe for ~9 days; Localize should name the region.
Transaction failures spike	A payments incident over the last ~7 days; graded urgent; fee revenue follows down.
Loan demand spikes	Applications surge for ~10 days and approvals follow; no fault in the funnel.
Revenue falls — one region	Card spend in Asia collapses; revenue falls with no cause inside the onboarding funnel.
New account signups slow	Fewer openings over the last ~10 days, nothing wrong downstream.
Noise only — nothing planted	Variance without a level change. The engine must not report an anomaly — the false-positive check.

Two modes

Mode	Path	Speed	Plants a scenario?
Slow	POST /events/simulate/plant → regenerate ~4,000 customers in Postgres → full warehouse re-extract	~5–6 minutes	yes, in the real facts
Fast	POST /events/simulate mode:"fast" → pipeline /dev/seed → bronze straight to ClickHouse → transforms in place → scoped /intelligence/rescore	~15–30 seconds	yes — the template's <i>effect</i> modifiers are translated into /dev/seed 's rate / window / segment knobs

Fast mode is the demo path. After a fast plant the KPI card moves on the dashboard within ~10–30s (a 10-second WebSocket poll nudges a refetch), and the chatbot reflects it ~30–75s later once the scoped re-score completes.

The end-to-end demo

1. **Once:** click **Reset demo data** (~35s). The dashboard reads a clean, flat baseline.
2. Show the baseline — every KPI "no material movement".
3. **Fast mode ON**, pick e.g. "*Transaction failures spike*", **Run Simulation** (~25s).
4. The Transaction Failure Rate card jumps (measured: ~1.4% → ~9% in the 7-day window).

5. ~1 minute later, ask the chatbot "*why did the transaction failure rate rise?*" — it detects ("rose ~596%, urgent"), localises (payment transactions, named branches), gives the trust verdict and evidence, and recommends an action with an owner.

The four brief scenarios

All four run from one seeded run and show on the dashboard
(docs/SOLUTION.md §"The four demo scenarios"):

- 1. **Multi-factor movement** — revenue drops; the agent walks the chain and shows it was a KYC completion leak in one segment plus a transaction-failure rise, with contributions.
- 2. **Low confidence** — a spike the Trust Gate suspects is a glitch, but the calendar shows a real campaign; the engine abstains and names the one cheapest check.
- 3. **Sparse history** — a newly launched product with a few days of data returns a wide, caveated range, not a false anomaly.
- 4. **Role entitlement** — the same KYC insight shows revenue-at-risk to the CFO and not to Ops (where it also cannot be back-computed); an unauthorized-access burst is flagged urgent and isolated.

Configuration reference

Where configuration lives

File	Committed?	Consumed by	Needed to start?
.env (repo root)	.env.example only	Compose \${VAR} interpolation; vllm-server reads it directly	No — every var has a default
dashboard/.env.local	No — gitignored, no template	env_file: on dashboard	Yes — compose errors out
nexabank/backend/.env	No — gitignored, no template	env_file: on nexabank-backend	Yes — compose errors out
rbac.json	Yes	dashboard + Analytics API	Yes — this is who can see what, and the persona registry
contracts/*.yaml	Yes	the intelligence layer	Yes — the KPI definitions

Key environment variables (root .env / compose)

Variable	Default	What it does
INTELLIGENCE_LLM	1	0 turns the LLM narrator off entirely (no number changes)

Variable	Default	What it does
VLLM_URL	http://ollama:11434/v1	OpenAI-compatible base URL for the narrator (name is historical)
GROQ_API_KEY / GROQ_MODEL	(unset)	enables the Groq provider in the chat model switcher
EXTRACT_API_TOKEN	local-dev-extract-token	shared secret for NexaBank's /api/extract/*; endpoints 503 without it
ENABLE_DEV_SEED	1	gates the pipeline's /dev/seed and /dev/seed/purge — unmistakably a dev tool
PIPELINE_EXTRACT_INTERVAL_S / PIPELINE_TRANSFORM_INTERVAL_S	600	the scheduled extract / transform loop period
PIPELINE_RUN_ON_START	0	run one full extract+transform on pipeline startup
INTELLIGENCE_WINDOW_DAYS	7	the default scored window
INTELLIGENCE_DATASET	seeded	seeded allows localising on otherwise-fabricated dims; live does not
NEXT_MODE	production	development compiles each Next.js route on first visit (17–24 s); production builds ahead
DEPLOYMENT_MODE	CLOUD	ON_PREM switches ingestion to a direct insert and pins to one tenant

~40 more intelligence knobs are prefixed `INTEL_ / INTELLIGENCE_`, all with sensible defaults, all documented in `api/intelligence/config.py`. Nothing in that layer is hardcoded.

What needs a rebuild after you edit what

The single most common source of "my change did nothing".

You edited	What to run
<code>api/</code> (except <code>analytics-api</code> and <code>intelligence</code> , which bind-mount <code>./api</code>)	usually nothing — but see the <code>__pycache__</code> trap below

You edited	What to run
<code>ingestion/, pipeline/, warehouse/, contracts/</code>	<code>docker compose up -d --build <service></code> — their source is baked in
<code>dashboard/src, nexabank/*/src</code>	bind-mounted, but Windows bind-mount watchers are unreliable: <code>docker compose restart <service></code>
<code>rbac.json</code>	live in the dashboard immediately (it bind-mounts the file); the Analytics API reads the copy baked into its image — <code>docker compose up -d --build analytics-api</code>
<code>warehouse/clickhouse/**/*.*.sql</code>	schema files auto-apply only to an empty volume . Add a numbered file to <code>warehouse/clickhouse/migrations/</code> and run <code>warehouse/migrate.py</code>
<code>.env</code>	<code>docker compose up -d</code> re-creates containers with the new values

Traps that have cost real time (from `docs/ARCHITECTURE.md`):

- **The Kafka fallback is silent.** Ingestion writing ClickHouse directly never crashes and never logs at a level anyone watches. `curl localhost:8000/health` → `ingest_path`.
- **A mounted Python dir can serve stale bytecode.** The `intelligence / analytics-api` containers bind-mount `./api` and write `__pycache__` back into it; Windows bind-mount mtimes are unreliable, so an edited module can keep running the old version through a full recreate. `rm -rf` the `__pycache__` dirs under `api/` when a change won't take.
- **`tsc --noEmit` passing proves the mount is current and nothing about the running process.**
- **`today()` in ClickHouse is server-local** — it once split the KPI card, the traffic chart and the rollup into three different days. The container is pinned `TZ=UTC`; use `toDate(now('UTC'))`.

Project structure

```

.
├── CLAUDE.md                the governing document – the brief, the rules,
the four-track split
├── docker-compose.yml      14 services + 2 opt-in profiles
├── Dockerfile              the shared Python image for six services
├── Dockerfile.ollama       builds the Ollama image with qwen2.5:3b baked in
├── rbac.json               roles + the persona registry (one tenant:
nexabank)
├── requirements.txt        Python deps (numpy/scipy for the stats;
statsmodels/psqueeze optional)
├── contracts/              the five KPI contracts + levers.yaml (the Decide
lever library)
│   ├── signups.yaml        kyc_completion_rate.yaml  loan_approval_volume.yaml
│   └── revenue.yaml        transaction_failure_rate.yaml  levers.yaml
└── nexabank/               the data source

```

```

|   |   | backend/                                Express 4 + Prisma 6 + ws (:5000)
|   |   |   | src/
|   |   |   |   | routes/                        events, extract, accounts, loans, transactions,
auth, pro
|   |   |   |   | middleware/eventTracker.ts    the producer-side taxonomy + fire-
and-forget forwarder
|   |   |   |   | simulate/                    generateFacts.ts (bulk fact generator) +
templates.ts (anomaly shapes)
|   |   |   |   | helper/                      journeyModel.ts, simulationBehavior.ts
|   |   |   |   | scripts/                    seedReferenceData, generateDemoData,
applyLifecycle, resetDemoData, plantMovement
|   |   |   |   | seedTenants.ts              idempotent tenant + system-admin + ledger-
account seed
|   |   |   | frontend/                        Next.js retail banking app (:3002) +
/admin/simulate console
|
|   |   | ingestion/main.py                    Ingestion API (:8000) - /events, /health, shape
validation, PII masking
|
|   |   | pipeline/                            kafka consumer + bronze→silver→gold transforms +
the extract
|   |   |   | worker.py                        kafka consumer → bronze.events
|   |   |   | service.py                      the scheduler + POST /refresh + POST /dev/seed
(:8003)
|   |   |   | extract/                        core_banking.py reference.py freshness.py
|   |   |   | transforms/                    silver_facts silver_events silver_sessions
gold_kpi gold_funnel
|   |   |   | taxonomy/                      the one canonicalisation vocabulary + rejects
|   |   |   |   | dev/seed.py                fast-mode ClickHouse-direct seeder (gated by
ENABLE_DEV_SEED)
|
|   |   | warehouse/
|   |   |   | migrate.py                      applies the layered DDL in bronze → silver →
gold order
|   |   |   |   | clickhouse/{bronze,silver,gold}/*.sql the authoritative layered DDL
|   |   |   |   |   | migrations/            post-baseline numbered deltas
|
|   |   | api/                                FastAPI service on :8001
|   |   |   | dashboard_api/main.py          the dashboard + /intelligence/* endpoints +
/ws/dashboard
|   |   |   | metric_api/                    the ONLY warehouse doorway - reads.py (named
reads), main.py (router), client.py
|   |   |   | contracts_loader.py            loads contracts/*.yaml; resolves kpi_id → gold
fundamentals
|   |   |   | middleware.py                  persona resolution + the Ops-hides-revenue
filter
|   |   |   | websocket_manager.py           per-tenant fan-out: Kafka tail + 10 s ClickHouse
poll
|   |   |   |   | intelligence/              the agent, tools, narrator, verifier, personas
|   |   |   |   |   | service.py            the scheduler loops (forecast batch +
investigation sweep)
|   |   |   |   | orchestrator.py           runs the six tools in order per KPI
|   |   |   |   | tools.py                  trust_gate · detect · localize · forecast ·
materiality · decide

```

```

|       |— loop.py  planner.py  the bounded query agent + rule/LLM planners
|       |— llm_client.py      OpenAI-compatible client (Ollama / Groq / vLLM),
bearer auth
|       |— stages/llm_narrator.py  the narrator prompt + persona-depth briefs
|       |— signal_store.py        one writer per Signal Store table
|       |— phrasing.py            the single place that decides how a figure reads
in English
|       |— personas.py  reader.py  config.py
|
|— dashboard/                                FinInsights dashboard (Next.js 15 / React 19,
:3001)
|   |— src/
|   |   |— app/(main)/                    dashboard, intelligence, admin, per-app routes
|   |   |— components/intel/              the agentic surface – ChatSurface, AnswerCards,
InsightRail
|   |   |— hooks/                          useDashboard (React Query + WebSocket),
useRealtimeEvents
|   |   |— lib/                            api.ts, feature-map.ts, ws-url.ts, rbac.ts
|
|— scripts/                                seed_data.py · run_intelligence_gates.py ·
verify_data_quality.py · taxonomy_probe.js
|— fixtures/planted_truth.json             the golden-scenario ground truth the Simulate
console writes
|— docs/                                  all documentation (see the map below)

```

Working on the code

The one rule

Run everything through Docker. `docker compose exec <service> ...` is the default for every check, query, type-check and script. A host `.venv` or host `node_modules` drifts from the image silently, and the container is what actually runs.

```

# Python – /app is the repo root
docker compose exec analytics-api python -c \
  "from api.contracts_loader import all_kpi_ids; print(all_kpi_ids())"

# ClickHouse
docker compose exec clickhouse clickhouse-client --password clickhouse \
  --query "SELECT date, fundamental, value FROM gold.kpi_daily \
    WHERE kpi_id='transaction_failure_rate' ORDER BY date DESC LIMIT 10"

# TypeScript, without touching host node_modules
docker compose exec dashboard npx tsc --noEmit
docker compose exec nexabank-backend npx tsc --noEmit
docker compose exec nexabank-frontend npx tsc --noEmit

# Warehouse migrations
docker compose exec -T ingestion-api python warehouse/migrate.py --status # list
docker compose exec -T ingestion-api python warehouse/migrate.py #

```

```
apply pending
```

```
# Logs
```

```
docker compose logs -f --tail=100 processor-worker
```

```
docker compose logs -f ingestion-api analytics-api intelligence pipeline
```

Operator tooling (the **tools** profile)

Dev deps and **node** live only in the **tools** image. Its default command is **verify_data_quality.py**; override it to run anything:

```
docker compose --profile tools run --rm tools python scripts/seed_data.py --
scenario all
docker compose --profile tools run --rm tools python
scripts/verify_data_quality.py
docker compose --profile tools run --rm tools python
scripts/run_intelligence_gates.py
```

Status note. **scripts/verify_data_quality.py** and **scripts/run_intelligence_gates.py** are **Round-1 scripts** that still reference the old single **feature_intelligence** database and have not been rebuilt for **bronze** / **silver** / **gold**. Round-2 pipeline verification (the four \$10 scenarios + a full **down -v** determinism diff) is recorded in **docs/execution/PHASE_7_REPORT.md** (gitignored). **pipeline/taxonomy/test_aliases.py** is the live canonicalisation test.

Adding a tracked event

Event names are three lowercase dot-separated segments (**<page>.<feature>.<status>**). The taxonomy is implemented at the producer (Node), and canonicalised once in Silver. Silver **rejects** an unresolvable name to the dead-letter table with a reason — it no longer silently coerces.

1. Emit it three-part: **trackEvent("loans.emi_estimator.success", ...)** on the backend.
2. Add any alias to the canonicalisation vocabulary in **pipeline/taxonomy/**.
3. Prove it — check the dead-letter table and **gold.kpi_daily**, don't read the code.

Adding a KPI

Write a YAML file in **contracts/** declaring the definition, grain, additive fundamentals, sliceable dimensions, detection thresholds, strategic weight, access rules, and the closed list of levers. Then teach **pipeline/transforms/gold_kpi.py** to roll up its fundamentals into **gold.kpi_daily**. No changes to the orchestration layer — the planner reads the contract catalogue at runtime.

Building an intelligence stage

Read data **only** through the Metric API. Write findings to the Signal Store. **Derive** ids, do not generate them. Tag every produced number with its engine. Never let a model produce a figure that reaches a reader unverified.

API surface

NexaBank backend — :5000/api

Endpoint	Purpose
POST /auth/login · POST /auth/register · GET /auth/profile	JWT-in-httpOnly-cookie auth
GET /extract/{entity}	token-guarded daily extract — <code>transactions</code> , <code>loan_applications</code> , <code>accounts</code> , <code>customers</code> , <code>campaigns</code> , <code>branches</code> ; keyset pagination
POST /events/simulate	per-user journey simulation (<code>`mode: "slow"</code>
POST /events/simulate/plant	regenerate the banking facts with an anomaly template applied (slow)
POST /events/simulate/reset	full wipe + fresh ClickHouse-direct baseline (<code>purge_full</code>)
GET /events/simulate/catalog	the route/event vocabulary the console may target
GET /health/forwarder	telemetry-forwarder success/failure counts

Ingestion API — :8000

Endpoint	Purpose
POST /events	the event envelope → Kafka (202); shape-validated, PII-masked, deduplicated by <code>event_id</code>
POST /events/seed/fast · /events/seed/fast/purge	legacy fast-seed (superseded by the pipeline's <code>/dev/seed</code>)
GET /health	<code>`ingest_path: kafka</code>

Pipeline service — :8003

Endpoint	Purpose
POST /refresh?full=<bool>	run extract + all transforms synchronously now
POST /dev/seed	fast ClickHouse-direct seed (users, days, behavior, <code>purge_first</code> , <code>purge_full</code> , <code>create_accounts</code>)
POST /dev/seed/purge	purge fast-seeded rows (<code>purge_full</code> widens to every fact row)
GET /health	last extract / transform / error

Analytics API — :8001

Group	Endpoints
Health / tenants	GET /health · /deployment/info · /tenants
Dashboard KPIs	GET /metrics/kpi · /metrics/traffic · /funnels
Metric API (named reads)	GET /metric/kpi/total · /metric/kpi/series · /metric/kpi/by_dim · /metric/kpi/cells · /metric/kpi/cell_deltas · /metric/funnel · /metric/freshness · /metric/watermark · /metric/calendar · /metric/campaigns · /metric/dimensions · /metric/dedup_counts · /metric/discover
Intelligence reads	GET /intelligence/insight · /insights · /recommendations · /series · /sources · /telemetry · /personas · /rail
Intelligence actions	POST /intelligence/ask · /intelligence/ask/stream (SSE) · /intelligence/rescore?kpi_id=...&days=... · /intelligence/outcome
Real-time	WS /ws/dashboard/{tenant_id} — METRICS_UPDATE (10 s poll) + REALTIME_EVENT (Kafka tail)

Known limitations

Stated plainly, because that is more useful than pretending otherwise.

- **RBAC is not a production security boundary.** The Analytics API trusts the role / email / app-scope headers as identity, the dashboard's browser-side layer sets them, and `:8001` is published to the host. Anyone who can reach it can assert any role. Persona resolution being server-side narrows nothing when the identity it resolves from is asserted, not proven.
- **Neither the ingestion nor the pipeline API authenticates anything.** `POST /events` on `:8000` and `/dev/seed` on `:8003` are open to any caller who can reach them.
- **NEXTAUTH_SECRET is a shared default hardcoded in docker-compose.yml.** `Compose environment:` overrides `env_file:`, so what you write in `.env.local` is ignored at runtime. Rotate it before this goes anywhere real.
- **Geography, device and channel are fabricated on the live path** — drawn once per session from a weighted table, not measured. The envelope declares this in `metadata._simulated`, the dashboard badges anything built on them, and Localize refuses to slice them outside the seeded dataset — but they are dice rolls.
- **Revenue is modelled, not measured.** There is no money column on a transaction; `fee_revenue` is a real fee schedule applied to real amounts, `interest_accrued` is straight-line, and `pro_revenue` is a headcount times a fixed price. The contract carries a qualifier the narrator repeats verbatim, and currency units are not reconciled across contracts.
- **The slow /plant path takes ~5–6 minutes**, dominated by a full re-extract of the whole transaction history, and `bronze.core_banking` grows on every full re-extract (a Track B item — see `docs/audit/TRACK_A_B_SYNC.md` §8c). Use fast mode + the Reset button for demos.
- **The Round-1 operator scripts are not rebuilt** for the layered warehouse (see the status note above).

- **The Kafka topic has one partition.** The producer keys on tenant and the worker handles rebalancing, but none of that is exercised. Scope any throughput claim accordingly.
 - **Windows bind-mount staleness** bites twice — TypeScript watchers, and Python `__pycache__` under a mounted `./api`. Restart / clear cache before concluding a change is wrong.
 - **Scoped out on purpose** (not simply unbuilt): full synthetic-control causal inference, learning from recorded outcomes, any autonomy that acts without a human signature, open-ended metric discovery beyond the conservative automatic tier, and per-series trained forecasters (which would break the zero-training-data property).
-

Troubleshooting

`docker compose up` fails immediately with an env-file error.

You have not created `dashboard/.env.local` and/or `nexabank/backend/.env`. Neither is in the repo. See [Installation step 2](#).

Every chart is empty and every number is zero.

The stack seeds nothing on its own. Run [Loading data](#). Then check `curl localhost:8000/health` — if `ingest_path` is `clickhouse_fallback`, events are landing but Kafka is out of the path; once `broker` is healthy, `docker compose restart ingestion-api`.

The `ollama` container fails to start / `docker compose up` errors on `ollama`.

No NVIDIA GPU or no NVIDIA Container Toolkit. Either delete the `deploy:` block from the `ollama` service in `docker-compose.yml` (it then runs on CPU), or set `INTELLIGENCE_LLM=0` and it does not matter, or use the Groq provider. The rest of the stack is unaffected.

Simulation returns HTTP 409 "No branches for this tenant".

Run `seedReferenceData.ts` first — branches are reference data and skipping them strips `region` out of every KPI that localises on it.

Simulation returns HTTP 409 "No existing customers to simulate".

A run generates activity for customers the bank already has. Tick **"Create accounts"** on the first run, or use the **Reset demo data** button which mints a population.

The extract endpoints return 503.

`EXTRACT_API_TOKEN` is unset on `nexabank-backend`. Set it in the root `.env` and restart.

I edited a Python file and nothing changed.

`ingestion-api` / `pipeline` / `processor-worker` bake their source in — `docker compose up -d --build <service>`. `analytics-api` / `intelligence` bind-mount `./api` but can serve stale `.pyc` through a Windows mount — `rm -rf` the `__pycache__` dirs under `api/` and restart.

I edited a TypeScript file and nothing changed.

Bind-mounted, but the watchers miss writes through a Windows bind mount. `docker compose restart <service>`. `tsc --noEmit` passing proves the mount is current and nothing about the running process.

The dashboard sends me to `/unauthorized` after signing in.

Your email is not in `rbac.json` (under `super_admins` or `app_admins.nexabank`), or — if using Google — you did not add yourself as a Test user on the OAuth consent screen.

`curl localhost:8001/docs` returns 403.

The RBAC middleware exempts only a handful of paths and a browser sends no role header. `GET /health` and `GET /deployment/info` are the open liveness probes.

The chatbot says "no material movement" right after I planted a scenario.

The scheduled sweep runs every ~3 minutes. Fast mode fires a scoped `/intelligence/rescore` automatically (~30–75 s); slow `/plant` does not, so wait for the sweep or call `POST /intelligence/rescore?tenants=nexabank&days=7` yourself.

A planted fast-mode movement barely shows on the KPI card.

The fast seed is small next to any real extracted history. Click **Reset demo data** first so the baseline is fast-seeded too, then the plant dominates its window.

`vllm-server` restart-loops with `$'\r': command not found`.

`vllm_entrpoint.sh` was checked out CRLF. `.gitattributes` prevents this — `git add --renormalize .` or re-clone.

FAQ

Do I need a GPU? No. The narrator is optional; the deterministic template covers every demo scenario and is bit-exact. A GPU only makes the local Ollama narrator faster; Groq needs none.

Do all 14 services need to run? Yes for a full demo. `vllm-server` and `tools` are opt-in profiles. If you have no GPU, drop the `ollama deploy: block` (CPU) or run `INTELLIGENCE_LLM=0`.

Does the AI ever make up a number? It is designed not to be able to. Every figure in an `/intelligence/*` narrative must match a stored row within tolerance; an unverifiable draft is stripped to the headline or the answer abstains.

Why does a Trust Gate failure still produce output? "The data is corrupted" is a genuine, useful answer. The run terminates the business path and issues an engineering note — the whole point is that a data defect never gets narrated as a business finding.

How do I point this at my own application instead of NexaBank? Emit events that satisfy the envelope, add your tenant id to the maps that must agree (`api/`, the dashboard feature map, `rbac.json`, `INTELLIGENCE_TENANTS`), then write a KPI contract for anything you want full investigation depth on.

Can I make my own account a NexaBank admin? Set `SYSTEM_PASSWORD` in `nexabank/backend/.env` before the first boot and sign in as `SYSTEM_EMAIL`, or flip `Customer.role` to `ADMIN` in Postgres directly. There is no promotion script.

Are there secrets in the repo? No `.env` file is tracked, only `.env.example`. Two things warrant attention: the `NEXTAUTH_SECRET` default in `docker-compose.yml`, and `rbac.json`, which ships with the maintainers' real email addresses. If you fork this, clear `rbac.json` and rotate the secret. If you add a `GROQ_API_KEY`, it lives only in your gitignored `.env`.

Documentation map

Markdown lives in `docs/` and nowhere else. `CLAUDE.md` and `README.md` are the only root markdown files.

Document	What it answers
<code>CLAUDE.md</code>	The brief, the invariants, the KPIs and personas, the tech, the four-track split. Governs everything.
<code>docs/SOLUTION.md</code>	What the product does and why, in plain English.
<code>docs/ARCHITECTURE.md</code>	How the pieces fit, how data flows, and the traps that have cost real time.
<code>docs/DATA_MODEL.md</code>	Bronze / Silver / Gold — what lives in each layer and who may read it; the Signal Store.
<code>docs/INTELLIGENCE.md</code>	The agent, its six tools, the narrator, the verifier, entitlement.
<code>docs/INTERFACES/</code>	The frozen contracts — <code>event_envelope.md</code> , <code>metric_api.md</code> , <code>kpi_rollups.md</code> .
<code>docs/audit/</code>	The Track B rebuild — the audit, the 8-phase plan, and <code>TRACK_A_B_SYNC.md</code> (the A $\rightleftharpoons$ B interface contract).
<code>contracts/*.yaml</code>	One KPI / semantic contract per KPI, plus <code>levers.yaml</code> . The narrator uses these and nothing else.

Maintainers and licence

Maintainer	GitHub
Abhishek Kumawat	@abhishekkumawat-47
Omesh Mehta	@Omesh2004
Vinod Singh Rathore	@vin0drath0re

No licence file is currently present. All rights reserved by the maintainers until one is added.